



DRAFT MASTER PLAN · AWAITING APPROVAL

PostForge

A template-driven Instagram content engine.
PackRadar is its first consumer — not its subject.

PREPARED BY	DATE	STATUS	TAG
BRAINSTORMER · PROJECT ARCHITECT	6 AUGUST 2026	DRAFT — NOT YET DECOMPOSED	POSTFORGE

How to read this. Sections A–C are the challenge: what your examples imply, what the Instagram API actually permits (verified against Meta's docs, not from memory), and the twelve gaps I found in the original idea. Sections D–I are the plan that follows from your answers. Nothing has been created in DevChain yet — that happens on your approval.

CONTENTS

- | | |
|--|---|
| A. What the twelve examples imply | F. Domain model & pipeline |
| B. Verified platform constraints | G. Stack |
| C. Twelve challenges to the model | H. Phase breakdown |
| D. Decisions taken | I. Assumptions, risks & open questions |
| E. Architecture — Engine vs. Pack | |

A What the twelve examples imply

Read from [examples/](#). Two clean families emerge — event-driven alerts and scheduled digests — with opposite timing requirements. The rule model has to serve both.

EXAMPLE	ARCHETYPE	FORMAT	TRIGGER	DATA DEPENDENCY
01-launch	Brand card	1:1	Manual / once	Store count only
02-drop	Restock alert	1:1	API push (event)	Store, product, price, stock, last-sellout duration, product photo
03 + 04	Intel report	1:1 × 6	Scheduled (weekly)	14-day aggregate; per-store price table
05-meme	Meme + screenshot	1:1	Scheduled / manual	Live store-page screenshot
06-story-live	In-stock now	9:16	Scheduled (daily)	Top-N in stock + link sticker
07-reel-cover	Reel cover	9:16	Manual	Video
08-sellout	Stat card	1:1	Scheduled (daily T+1)	Yesterday's sellout event
09-story-poll	Poll	9:16	Scheduled (weekly)	4 trending sets + poll sticker
10-price-drop	Price-drop alert	1:1	API push (event)	Old/new price, %, rank across 88 stores
11-ugc	UGC feature	1:1	Manual (inbound)	Follower photo + handle
12-weekly	Stat grid	1:1	Scheduled (weekly)	4 weekly KPIs

DESIGN SYSTEM ALREADY PRESENT

The twelve are visually coherent: near-black [#0A0F14](#), signal green [#2EE66B](#), mint [#C8F5DC](#), mono + geometric sans, hairline rules, occasional full-green inversion (03, 12). That is a token set, and it should be extracted before any layout is built.

B Verified platform constraints

Checked against Meta's content-publishing documentation on 6 Aug 2026 — because two of your twelve examples turn out not to be automatable at all.

CONSTRAINT	CONSEQUENCE FOR THIS BUILD
100 API posts / rolling 24 h per account; a carousel counts as 1	Ample for planned volume, but the engine still needs a hard daily cap and a minimum gap between posts
Media must be at a publicly reachable URL — Meta cURLs it	Forces a public blob store. No local files, no signed-private URLs
Requires IG professional account linked to a Facebook Page, Page Publishing Authorization, instagram_basic + instagram_content_publish	Meta App Review is the schedule risk — see Risks
Carousel max 10 items, all cropped to the first item's aspect ratio	Carousel builder must enforce one aspect across all slides at render time

Interactive stickers — poll, link, quiz, question — cannot be published via the API

Kills full automation of 09-story-poll and 06-story-live. Every third-party scheduler falls back to notification publishing

THE ONE THAT HURTS

06-story-live reads "link sticker goes here" — and the link sticker is the **only** mechanism by which an Instagram story sends traffic to packradar.app. It cannot be applied by any API. Assisted publishing is therefore not an optional convenience; without it the entire story funnel has no call to action.

c Twelve challenges to the model

Each states the problem, then the resolution now folded into the plan.

1 Two templates die on contact with the API BLOCKED

09 needs a poll sticker, 06 needs a link sticker. Neither is API-publishable.

RESOLUTION · Every rendition carries `publish_mode: auto | assisted`. Assisted renders the asset, pushes it to your phone with caption and link pre-formatted; you add the sticker in-app and share.

2 The cancel window fights your own data — the largest hole

Your own copy: "sold out in 41 min. Move." and "18 min fastest sellout." A restock alert that waits 15 minutes in an approval queue can publish stock that no longer exists. For a brand called a radar, that is the fastest possible way to lose trust.

RESOLUTION · A mandatory **revalidate gate** after approval and before publish: re-read the signal, diff against what was rendered, and abort or swap to a variant if the fact moved. Per-template freshness TTL — restock 5 min, weekly report 7 days.

3 "If no mentions, post after X minutes" — seen where?

Nobody cancels what they never saw, and a five-minute window is meaningless over email.

RESOLUTION · Telegram bot with inline Approve / Cancel / Edit / Snooze; the web dashboard remains system of record.

4 Template chaining becomes a graph you will regret

"template2 if template1 skipped; template3 is fallback for others" are pointers between templates. Add a fourth and you have cycles, ambiguous precedence, and no way to answer "what posts at 18:00?".

RESOLUTION · Invert to **Slots**. A slot is a scheduled publishing opportunity holding an *ordered candidate list*; evaluate top-down, first template whose condition passes wins, none pass → slot logged empty. Fallback is list position, not a reference. Cycles become impossible.

5 Missing entirely: dedupe and cooldown

Your data repeats — the same booster box stays in stock for days. With no memory you post Prismatic Evolutions five days running and look broken.

RESOLUTION · Cooldown per `(template, entity_key)`, global minimum gap, daily cap per account, and no re-alert of the same SKU/store within N hours.

6 A story is not a crosspost of the feed post

06 and 09 are genuinely different layouts with different content density from 02 and 10. Auto-scaling 1:1 into 9:16 produces letterboxed filler.

RESOLUTION · A template is one logical content unit with *explicitly authored renditions*. It offers a story only if a story rendition exists.

7 Where do product photos come from?

02 and 11 ship with empty image slots. Scraping store photos raises a copyright question, and Instagram needs a public URL regardless.

RESOLUTION · A curated asset library keyed by product/set, plus a mandatory typographic fallback layout when no photo exists — your strongest cards (08, 10) already carry no imagery. Never a broken image.

8 The reel is a different product DEFERRED

07 implies video: script, timeline, audio, render — a second rendering stack.

RESOLUTION · Out of v1, to backlog.

9 The meme needs a screenshot pipeline DEFERRED

05 embeds a live store page: headless browser plus per-store selectors that break constantly. High maintenance, low volume.

RESOLUTION · Backlog, or keep manual.

10 Captions can contradict the image

An LLM writing "under 900 lei" over a card rendering 925 lei publishes a false claim.

RESOLUTION · Captions derive deterministically from the same payload as the image. An LLM may write only the hook line, validated against that payload before the item can enter the queue.

11 The data-source contract is unknown

Push or pull? Existing API or none?

RESOLUTION · Dissolved by the connector abstraction (Section E) — the engine needs no knowledge of PackRadar's schema. It only gates the Phase 5 pack.

12 Token lifecycle is unhandled

Long-lived tokens expire in roughly 60 days. With no refresh job the app dies silently two months after launch and your audience tells you before your logs do.

RESOLUTION · Scheduled refresh plus an expiry-warning alert, in Phase 2.

D Decisions taken

QUESTION	YOUR ANSWER	ARCHITECTURAL CONSEQUENCE
Scope	Not internal. A product used for PackRadar first, possibly generalised to market later	The single most consequential answer. No PackRadar concept may exist in engine code. <code>workspace_id</code> on every row from day one; no signup or billing in v1
Approval surface	Telegram bot	Inline Approve / Cancel / Edit / Snooze; fast enough for a 5-minute window on a phone

Approval policies — your three modes, named

POLICY	BEHAVIOUR
auto	Skip review entirely; revalidate, then publish
cancel_window	Render → notify → wait N minutes → publish unless cancelled
require_approval	Never publishes without an explicit yes; expires after N minutes

NON-NEGOTIABLE

The revalidate gate sits after approval and before publish, in every mode including `auto`. It is the mechanism that makes a delayed post safe, and it was the single largest omission in the original description.

G Stack

Recommended; assumed unless you object.

CONCERN	CHOICE	REASON
App	Next.js App Router on Vercel	Dashboard, API triggers and cron in one deployable
Database	Postgres (Neon, via Marketplace)	Relational model with JSONB payloads
Media hosting	Vercel Blob — public store	Mandatory: Meta cURLs the media, so the URL must be public
Rendering	Satori / <code>@vercel/og</code>	No browser, ~100 ms, runs in a Function. All 10 layouts fit its flexbox subset
Scheduling	Vercel Cron	Slot ticks, token refresh, cancel-window sweeper
Notifications	Telegram Bot API	Instant, mobile, inline buttons, no app to build
Screenshots	Headless Chromium	Only if the meme template is later revived

DELIBERATELY REJECTED · VERCEL WORKFLOW FOR THE CANCEL WINDOW

It fits the shape — sleep N minutes, then continue — but the dashboard must query and mutate that same state anyway (list pending, cancel, edit). Durable-workflow state would duplicate the database and fight every Cancel click. A database row plus a cron sweeper is the simpler correct answer here.

H Phase breakdown

Risk-first ordering. The two things that can kill the project — rendering fidelity and Meta publishing — are proven in P1 and P2, not discovered in P5.

PHASE	OUTCOME	SHIPS	WHY HERE
P0	Foundations	<code>docs/</code> , <code>development-standards.md</code> , repo scaffold, schema, workspace + account seed	SOP requires documentation before decomposition; executed directly by the Architect

PHASE	OUTCOME	SHIPS	WHY HERE
P1	Render vertical slice	Design tokens extracted from <code>examples/</code> , layout component contract, Satori pipeline, one real layout (<code>02-drop</code>) rendering live data to a public URL, preview page	Proves the visual system survives real data — long store names, four-digit prices, missing photos
P2	Publish for real	IG OAuth + token refresh, publish adapter (image / carousel / story), ContentItem state machine, revalidate gate, rate limit + daily cap	Proves the hardest external dependency early. Meta App Review must begin during P0
P3	Rules & triggers	Slots with rule + timezone, condition evaluation, candidate fallback, cooldown / dedupe by <code>entity_key</code> , minimum gap, API trigger endpoint with schema validation	The brain. At the end of P3 the account can run itself
P4	Approval layer	Telegram bot with preview and inline actions, cancel-window sweeper, assisted-publish handoff, expiry semantics	Needs P2 and P3 to exist before it means anything
P5	PackRadar pack	Remaining 9 layouts, carousel multi-slide builder, asset library + typographic fallback, caption and hashtag sets, connector config	Pure pack work — requires no engine change if P1–P4 are right. This is the test of Section E
P6	Operations	Audit log with data snapshot, failure alerts, token-expiry warnings, dashboard polish, dry-run mode	Hardening

Backlog — passes the value gate, not v1

Reels / video (`07`) · meme screenshot pipeline (`05`) · UGC consent flow (`11`) · visual layout editor · multi-tenant signup and billing · additional platforms, for which the `platform` discriminator already exists.

I Assumptions, risks & open questions

Assumptions — correct any and I will adjust

	ASSUMPTION	RATIONALE
1	Caption language is English	Your examples are English despite a Romanian market
2	<code>require_approval</code> expiry drops the post rather than publishing it	Safer default; overridable per template
3	Product photos come from a curated library, never scraped at publish time	Copyright exposure, and Instagram needs a stable public URL
4	Captions are deterministic; LLM writes the hook line only, validated against the payload	Prevents a caption claiming 899 over a card rendering 925
5	One Instagram account is configured in v1; the model supports N	Matches your scope answer without pre-building tenancy UI

Risks

RISK	SEVERITY	MITIGATION
Meta App Review latency for <code>instagram_content_publish</code>	High — schedule	Start during P0, in parallel with code. This is the long pole, and it is not an engineering task
Satori's CSS subset cannot express a layout	Medium	Proven against the hardest layout in P1, before nine more are committed to
Stale data published during an approval delay	High — brand	Revalidate gate; freshness TTL per template
PackRadar concepts leaking into engine code	Medium — future	CI grep guard; P5 serves as the proof that the split held

Open questions — needed before P1 closes

- Q1 Does PackRadar expose an API today? Endpoints and auth, or "not yet" — either answer is workable; it gates only P5.
- Q2 Do you already have an Instagram Business account, a linked Facebook Page, and a Meta app? If not, this starts today in parallel — App Review is the schedule risk, not the code.
- Q3 Are the example JPGs the final design direction, or a first pass I should treat as loose reference?
- Q4 Any objection to the stack in Section G?

On approval I will create `docs/` and `docs/development-standards.md` per SOP §11, then decompose P0–P6 into Phase Epics, Backlog Epics and sub-epics in DevChain under the tag `postforge`. No application code is written by this role — a Coder agent picks that up.

Sources · Meta — Instagram Content Publishing · Vista Social — stickers and links are not publishable via API